FeaturesCompose

Paging3 Pagination from Remote API & Local Cache

- Paging 3 library in Android is used for handling large data sets in a more efficient way, particularly when working with RecyclerViews.
- It is part of the **Android Jetpack library** and provides a set of components to load and display large data sets incrementally.

You should consider using Paging 3 in Android under the following scenarios:

Large Datasets: When dealing with a **large amount of data** that cannot be loaded entirely into memory at once.

Efficient Loading: When you want to **load and display data incrementally**, providing a smoother user experience.

Network Requests: When loading data from a network source, Paging 3 helps in making **efficient network requests and managing the data.**

Database Paging: When working with local databases (e.g., Room database), Paging 3 can **efficiently load data from the database.**

RecyclerView Integration: When you want to seamlessly integrate with RecyclerView to efficiently display paginated data.

Here's a basic example of when you might use Paging 3:

```
// Create a PagingConfig
val pagingConfig = PagingConfig(pageSize = 20, enablePlaceholders = false)

// Create a Pager
val pager: Pager<Int, YourDataType> = Pager(
    config = pagingConfig,
    pagingSourceFactory = { YourPagingSource() }
)

// Create a LiveData to observe
val pagedLiveData: LiveData<PagingData<YourDataType>> = pager.liveData
```

In this example, YourPagingSource would be a class that extends PagingSource, responsible for loading data in chunks.

Retrofit

Retrofit is a popular and widely-used library in the Android development ecosystem for making **network requests**, particularly when dealing with **APIs**.

Reasons to use Retrofit in Android apps:

Simplicity and Readability: Retrofit simplifies the process of making API requests by providing a **clean and intuitive syntax**. It uses **annotations** to define the API interface, making the code more **readable and maintainable.**

Type-Safe HTTP Calls: Retrofit generates a type-safe API interface based on the defined endpoints and methods. This* *helps catch errors at compile-time rather than runtime**, reducing the chances of bugs in your network code.

Automatic Serialization/Deserialization: Retrofit supports automatic conversion of JSON responses to Java or Kotlin objects using converters like Gson or Moshi. This eliminates the need for manual parsing and streamlines the handling of data.

Customization and Extensibility: Retrofit is highly customizable. You can configure it to use different converters, interceptors, and other features to suit your specific needs. This flexibility is valuable when dealing with various API requirements.

Asynchronous Programming Support: Retrofit supports asynchronous programming through integration with popular libraries like RxJava or Kotlin Coroutines. This allows you to handle network requests asynchronously, preventing them from blocking the main UI thread.

HTTP Request Methods and Headers: Retrofit supports various HTTP request methods (GET, POST, PUT, DELETE, etc.) and allows you to easily set headers for your requests. This is essential when working with RESTful APIs that require specific HTTP methods or headers.

Handling Query Parameters and Path Segments: Retrofit provides convenient ways to include query parameters and path segments in your API requests. This makes it easy to construct dynamic URLs based on user input or other runtime conditions.

Caching and Request Interceptors: Retrofit allows you to integrate caching mechanisms and request interceptors. This can be useful for optimizing network requests, handling authentication, or adding custom behavior to each request.

In summary, Retrofit is a powerful and versatile library that simplifies the process of interacting with APIs in Android applications. It provides a clean and efficient way to handle network requests, making the development process more straightforward and maintaining good code practices.

More about Retrofit:

1. Annotations:

   Retrofit uses annotations to define API endpoints and their parameters. For example, @GET, @POST, @Query, @Path, etc., are used to describe the HTTP methods, query parameters, and path parameters.

2. Converters:

Retrofit supports multiple converters, such as Gson and Moshi, for handling the serialization and deserialization of JSON data. You can choose the converter that best fits your project requirements.

3. Error Handling:

   Retrofit provides a straightforward way to handle errors by allowing you to define a callback or use RxJava or Kotlin Coroutines to manage success and error responses.

4. Dynamic URLs:

   You can create dynamic URLs by using placeholders in the endpoint paths and providing corresponding values during runtime. This is useful for scenarios where the API URL depends on user input or other dynamic factors.

5. Multipart Requests:

   Retrofit supports multipart requests, allowing you to send files along with other form data in a single request.

6. RxJava and Kotlin Coroutines Support:

   Retrofit seamlessly integrates with RxJava or Kotlin Coroutines, providing a convenient way to perform asynchronous programming when making network requests.

Request Interceptors:

A Request Interceptor in Retrofit is an interface that allows you to modify outgoing requests before they are sent to the server. It's a powerful feature that can be used for various purposes, such as adding headers, modifying request parameters, logging, or handling authentication.

```
val interceptor = object : Interceptor {
    override fun intercept(chain: Interceptor.Chain): Response {
        val originalRequest = chain.request()

        // Modify the request by adding headers, authentication tokens,
etc.
        val modifiedRequest = originalRequest.newBuilder()
            .addHeader("Authorization", "Bearer your_token")
            .build()

        // Proceed with the modified request
        return chain.proceed(modifiedRequest)
    }
}

val httpClient = OkHttpClient.Builder()
    .addInterceptor(interceptor)
    .build()

val retrofit = Retrofit.Builder()
    .baseUrl("https://api.example.com/")
    .client(httpClient)
```

```
    .addConverterFactory(GsonConverterFactory.create())
    .build()
```

In this example, the interceptor is used to add an "Authorization" header to every outgoing request. You can customize the interceptor to suit your specific needs, making it a versatile tool for handling various aspects of network requests. Interceptors are particularly useful for tasks like:

```
**Authentication:** Adding authentication headers or tokens to requests.
**Logging**: Logging information about requests and responses for
debugging purposes.
**Header Modification:** Adding, removing, or modifying headers in
requests.
**Dynamic Request Modification:** Dynamically modifying requests based on
certain conditions.
```

By using Request Interceptors, you can centralize common logic for outgoing requests, making your code more modular and maintainable.

Shimmer effected is added in this branch